

Relazione di Laboratorio 01

High Performance Computing

Federico Becchi

15 luglio 2026

1 Installazione e configurazione

1.1 Descrivere l'installazione eseguita, incluso il sistema operativo e il tipo di GPU

Laptop MacBook Air M3.

Eseguo il comando `sw_vers`:

- Architettura: Apple Silicon (ARM64)
- OS: macOS 26.3.1

Eseguo il comando `system_profiler SPDisplaysDataType`:

- Apple M3 con GPU integrata (10-core)

Questo PC utilizza un'architettura Apple Silicon, strutturata così:

- 8 Core CPU
- 10 Core GPU
- 16 Core IA

La memoria è detta *unificata* tra CPU e GPU: questo comporta che utilizzino lo stesso insieme di memoria. Inoltre questa architettura utilizza un set di API Metal, nel mio portatile versione 4.

Decido di utilizzare l'architettura con due nodi (Docker). Ho dovuto modificare l'architettura utilizzata dai due nodi e il percorso dei volumi:

```
platform: linux/amd64 # per processori ARM con emulatore Rosetta 2
volumes:
  - /Users/federicobecchi/Documents/HPC/lab/01:/root
```

Inoltre ho dovuto rimuovere la sezione `deploy` contenente le istruzioni per caricare i driver NVIDIA.

1.1.1 Avvio architettura

```
docker compose up -d
(base) federicobecchi@192 01 % docker compose up -d
WARN[0000] No services to build
[+] up 3/3
v Network 01_default Created                                0.0s
```

v Container wn2	Created
-----------------	---------

Ora si configura l'accesso SSH da **wn1** a **wn2**, in particolare si vuole evitare di inserire la password ad ogni connessione.

Accesso SSH a **wn1**:

```
(base) federicobecchi@192 01 % docker compose exec wn1 bash
```

Successo.

Utilizzando i permessi si dà la proprietà della cartella **/root** all'utente **root**:

```
root@wn1:~# chown root:root /root
```

Ora si assegnano i permessi a **/root**:

```
chmod 755 /root
```

Così **root** può scrivere e gli altri possono leggere/entrare.

1.1.2 Creazione chiavi SSH

Creazione chiavi con algoritmo RSA con nessuna passphrase, salvata dentro **~/.ssh/id_rsa**:

```
ssh-keygen -t rsa -P '' -f ~/.ssh/id_rsa
```

In output si ottengono due file:

- **id_rsa** → chiave privata
- **id_rsa.pub** → chiave pubblica

Abilitazione accesso automatico:

```
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
```

Protezione alla cartella delle chiavi:

```
chmod 600 ~/.ssh/authorized_keys
```

Solo il proprietario può leggere e scrivere, agli altri utenti l'accesso è proibito.

wn1 si autentica su **wn2** tramite una chiave crittografica RSA invece che con password, permettendo la comunicazione automatica tra nodi (fondamentale per MPI).

1.2 Esecuzione test PC

1.2.1 Raccolta informazioni sulla configurazione del nodo di calcolo

```
root@wn2:~# sh deviceQuery/CPU/lscpu.sh
```

L'output è molto lungo, quindi ne riporto un riassunto integrato anche dalle informazioni iniziali; tuttavia queste sono filtrate dalla configurazione Docker verso la CPU fisica: presumo che ci sia una vista impostata, lo deduco dalla mancanza della dimensione della cache e della frequenza della CPU.

- Architettura: **x86_64** (interessante)

- Core: 8
- Socket: 1
- Thread per core: 1
- Istruzioni SIMD: ASIMD + AES + SHA extensions
- Supporto crittografico hardware: sì

```
root@wn2:~# sh deviceQuery/GPU/deviceQuery.sh
wn2
deviceQuery/GPU/deviceQuery.sh: 6: nvidia-smi: not found
deviceQuery/GPU/deviceQuery.sh: 7: ./deviceQuery: Permission denied
```

Su questo PC non è presente una scheda video NVIDIA.

Utilizzo questo comando:

```
(base) federicobecchi@192 ~ % system_profiler SPDisplaysDataType
Graphics/Displays:
  Apple M3:
    Chipset Model: Apple M3
    Type: GPU
    Bus: Built-In
    Total Number of Cores: 10
    Vendor: Apple (0x106b)
    Metal Support: Metal 4
    Displays:
      Color LCD:
        Display Type: Built-in Liquid Retina Display
        Resolution: 2560 x 1664 Retina
        Main Display: Yes
        Mirror: Off
        Online: Yes
        Automatically Adjust Brightness: Yes
        Connection Type: Internal
```

```
ioreg -l | grep -i gpu # per un report approfondito
system_profiler SPMetalDataType # per informazioni su Metal (equivalente di CUDA in
architettura NVIDIA)
```

1.2.2 Determinare le prestazioni di picco in FLOPS

MacBook Air M3.

1 core CPU: frequenza massima $\times 2$ (FMA)* $\times N$ (SIMD)**

Frequenza ~ 4.06 GHz (valore tipico boost M3), stima presa da internet; l'informazione tecnica non l'ho trovata accessibile.

SIMD ~ 8 (single precision), $N = 2$.

La CPU Apple M3 supporta istruzioni SIMD ASIMD (NEON) a 128 bit; utilizzando dati in doppia precisione (64 bit) il fattore SIMD risulta $N = 2$. FMA = 2.

$$\text{FLOPS}_{\text{core}} = f \times 2 \times \text{SIMD} = 16.25 \text{ GFLOPS} \quad (1)$$

I nodi MPI sono implementati tramite container Docker (wn1, wn2) eseguiti sul medesimo MacBook Air M3. Essi rappresentano nodi logici del cluster, ma condividono lo stesso hardware fisico; pertanto le prestazioni teoriche non vengono moltiplicate per il numero di container.

1 nodo CPU: prestazioni di $1 \text{ core} \times \text{numero di core fisici (no hyper-threading)}$

$$\text{FLOPS}_{\text{CPU}} = 16.25 \times 8 = 129.92 \text{ GFLOPS} \quad (2)$$

GPU clock tipico stimato: $\sim 1.3 \text{ GHz}$.

1 core GPU: frequenza massima $\times 2 \text{ (FMA)} = 2.6 \text{ GHz}$

1 GPU: prestazioni di $1 \text{ core} \times \text{numero di core} = 2.6 \times 10 = 26 \text{ GHz}$

La GPU Apple M3 non è basata su architettura CUDA e non dispone di CUDA core. Le prestazioni sono quindi stimate utilizzando il modello semplificato basato su frequenza di clock e numero di core, secondo la formula fornita dal corso. La programmazione GPU avviene tramite API Metal.

2 Test eseguiti sul cluster di Ateneo

2.1 Sezione Cluster Ateneo

XXX

2.2 Raccolta informazioni sulla configurazione del nodo di calcolo

CPU: numero di core, frequenza di lavoro, dati su memoria e cache, presenza di istruzioni SIMD (AVX, AVX2, AVX512, ...) e FMA.

```
[federico.becchi@ui01 deviceQuery]$ sh CPU/lsCPU.slurm
ui01.hpc.unipr.it
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Byte Order:            Little Endian
CPU(s):                4
On-line CPU(s) list:   0-3
Thread(s) per core:    1
Core(s) per socket:    2
Socket(s):              2
NUMA node(s):          1
Vendor ID:              GenuineIntel
CPU family:             6
Model:                 85
Model name:             Intel(R) Xeon(R) Gold 6238R CPU @ 2.20GHz
Stepping:               7
CPU MHz:               2194.843
BogoMIPS:               4389.68
Hypervisor vendor:      VMware
Virtualization type:    full
L1d cache:              32K
L1i cache:              32K
L2 cache:               1024K
L3 cache:               39424K
NUMA node0 CPU(s):     0-3
```

```
Flags:          fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov
pat pse36 clflush mmx fxsr sse sse2 ss ht syscall nx pdpe1gb rdtscp lm
constant_tsc arch_perfmon nopl xtopology tsc_reliable nonstop_tsc eagerfpu pni
pclmulqdq ssse3 fma cx16 pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer
aes xsave avx f16c rdrand hypervisor lahf_lm abm 3dnowprefetch invpcid_single
ssbd ibrs ibpb stibp ibrs_enhanced fsgsbase tsc_adjust bmi1 avx2 smep bmi2 invpcid
avx512f avx512dq rdseed adx smap clflushopt clwb avx512cd avx512bw avx512vl
xsaveopt xsavec arat pku ospke md_clear spec_ctrl intel_stibp flush_l1d
arch_capabilities
```

GPU: CUDA driver version, dimensione memoria globale, memoria shared, numero di CUDA core, max clock rate.

```
[federico.becchi@ui01 deviceQuery]$ sh GPU/deviceQuery.slurm
module
ui01.hpc.unipr.it
GPU 0: Tesla P100-PCIE-12GB (UUID: GPU-f2a13511-29bd-fd51-4068-6beb0f13136d)
./deviceQuery Starting...

  CUDA Device Query (Runtime API) version (CUDA static linking)

Detected 1 CUDA Capable device(s)

Device 0: "Tesla P100-PCIE-12GB"
  CUDA Driver Version / Runtime Version          12.4 / 11.5
  CUDA Capability Major/Minor version number:    6.0
  Total amount of global memory:                 12187 MBytes (12778733568 bytes)
  (056) Multiprocessors, (064) CUDA Cores/MP:    3584 CUDA Cores
  GPU Max Clock rate:                            1329 MHz (1.33 GHz)
  Memory Clock rate:                             715 Mhz
  Memory Bus Width:                              3072-bit
  L2 Cache Size:                                 3145728 bytes
  Maximum Texture Dimension Size (x,y,z)         1D=(131072), 2D=(131072, 65536), 3D
    =(16384, 16384, 16384)
  Maximum Layered 1D Texture Size, (num) layers  1D=(32768), 2048 layers
  Maximum Layered 2D Texture Size, (num) layers  2D=(32768, 32768), 2048 layers
  Total amount of constant memory:               65536 bytes
  Total amount of shared memory per block:        49152 bytes
  Total shared memory per multiprocessor:         65536 bytes
  Total number of registers available per block:  65536
  Warp size:                                     32
  Maximum number of threads per multiprocessor:   2048
  Maximum number of threads per block:            1024
  Max dimension size of a thread block (x,y,z):  (1024, 1024, 64)
  Max dimension size of a grid size    (x,y,z):  (2147483647, 65535, 65535)
  Maximum memory pitch:                          2147483647 bytes
  Texture alignment:                             512 bytes
  Concurrent copy and kernel execution:           Yes with 2 copy engine(s)
  Run time limit on kernels:                      No
  Integrated GPU sharing Host Memory:             No
  Support host page-locked memory mapping:        Yes
  Alignment requirement for Surfaces:             Yes
  Device has ECC support:                        Enabled
  Device supports Unified Addressing (UVA):        Yes
  Device supports Managed Memory:                 Yes
  Device supports Compute Preemption:             Yes
  Supports Cooperative Kernel Launch:             Yes
  Supports MultiDevice Co-op Kernel Launch:       Yes
```

```

Device PCI Domain ID / Bus ID / location ID:  0 / 4 / 0
Compute Mode:
  < Default (multiple host threads can use ::cudaSetDevice() with device
    simultaneously) >

deviceQuery, CUDA Driver = CUDART, CUDA Driver Version = 12.4, CUDA Runtime Version =
  11.5, NumDevs = 1
Result = PASS

```

2.3 Determinare le prestazioni di picco in FLOPS

1 core CPU: frequenza massima $\times 2$ (FMA)* $\times N$ (SIMD)**

Il nodo analizzato è un Intel Xeon Gold 6238R con frequenza di 2.20 GHz e supporto alle istruzioni AVX, AVX2 e AVX512.

Il set SIMD massimo disponibile è AVX512, corrispondente a $N = 16$.

Prestazioni per core:

$$\text{FLOPS}_{\text{core}} = 2.20 \times 10^9 \times 2 \times 16 = 70.4 \text{ GFLOPS} \quad (3)$$

Prestazioni nodo:

$$\text{FLOPS}_{\text{CPU}} = 70.4 \times 4 = 281.6 \text{ GFLOPS} \quad (4)$$

1 nodo CPU: prestazioni di 1 core $\times$ numero di core fisici (no hyper-threading)

$$\text{FLOPS}_{\text{CPU}} = 70.4 \times 4 = 281.6 \text{ GFLOPS} \quad (5)$$

1 core GPU: frequenza massima $\times 2$ (FMA) = 1.33×2

1 GPU: prestazioni di 1 core $\times$ numero di core = $1.33 \times 2 \times 3584$